Praktikum 1 - Matakuliah Pilihan Web (API Development)
Program Studi: Teknik Informatika

Lakukan praktikum dibawah ini, dan buat screenshot untuk pembuktian mengerjakan setiap poin dengan mengisi tabel dibawah, kemudian tunjukan hasil akhir dari men-share repository github yang telah dibuat.

A. Setup Laravel dan Mysql

1. Buat project laravel baru dengan composer dengan nama laravel_api (Gunakan versi 12.x atau yang lebih baru)
2. Mempersiapkan database dan buat database baru bernama dblaravelapi
3. Update .env dan hubungkan dengan nama database dblaravelapi
4. Lakukan migrasi dengan menggunakan php artisan migrate
5. Pastikan laravel sudah terhubung dengan tabel dengan mengecek tabel yang dibuat laravel dalam mysql

B. Membuat Controller

1. Buatlah sebuah controller dengan nama Api/ProductController **php artisan make:controller Api/ProductController**
   (Pastikan ada di dalam folder Api)
   Pada bootstrap/app.php tambahkan setelah code web: __DIR api: __DIR__.'/../routes/api.php',

```
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
```

2. Tambahkan Routes baru di router/api.php
   Route::get('/products', [ProductController::class, 'index']);

(Apabila file api.php belum ada, clone web.php dan hapus routes “/”) 3. Test API dengan mengunjungi localhost:8000/api/products

C. Gunakan database dan Model

1. Pastikan database sudah terkoneksi dengan laravel.
2. Buatkan sebuah tabel dengan nama tblproducts dengan field (id, nama, deskripsi, foto, dan harga) untuk tipe data menyesuaikan.

3. Tambahkan minimal 10 data untuk produk dengan nama2 product elektronik seperti tv, kulkas, ac, dan lain sebagainya.
4. Buat model bernama Product **php artisan make:model Product**
5. Hubungkan model Product dengan tblproduct dengan mengisi pada class Product

```
class Product extends Model
{
    protected $table = 'tblproducts';
    protected $primaryKey = 'id';
    public $timestamps = false;

    // Field yang boleh diisi
    protected $fillable = [
        'nama',
        'deksripsi',
        'foto',
        'harga',
    ];

}
```

6. Pada Product controller ubah dalam fungsi index, dapatkan data dari model Product
   Dengan menambahkan $products = Product:all();
   Dan lakukan return data => $products

D. Membuat CRUD standar API Resource

1. Ubah api.php dengan api Resource yang sesuai standar laravel

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\ProductController;

// Route::get('/products', [ProductController::class, 'index']);

Route::apiResource('products', ProductController::class);
```

Dengan apiResource kamu tidak perlu menambahkan routes satu persatu untuk ProductController seperti create, update, delete, search dan view. Namun ada syarat penamaan fungsi agar routes ini bekerja yaitu **index, show, store, update, dan destroy.**

2. Ubah ProductController untuk fungsi-fungsi berikut **index, show, store, update, dan destroy.**

```php
// ◆ GET /products
public function index()
{
    $products = Product::all();

    return response()->json([
        'status' => true,
        'data' => $products
    ]);
}
```

Gambar 04: Fungsi Index pada ProductsController

```php
// ◆ GET /products/{id}
public function show($id)
{
    $product = Product::find($id);

    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }

    return response()->json([
        'status' => true,
        'data' => $product
    ]);
}
```

Gambar 05: Fungsi Show pada ProductsController

```php
// ◆ POST /products
public function store(Request $request)
{
    $request->validate([
        'nama' => 'required',
        'deksripsi' => 'required',
        'harga' => 'required|numeric'
    ]);

    $product = Product::create($request->all());

    return response()->json([
        'status' => true,
        'message' => 'Product berhasil ditambahkan',
        'data' => $product
    ]);
}
```

Gambar 06: Fungsi Store pada ProductsController

```php
// ◆ PUT /products/{id}
public function update(Request $request, $id)
{
    $product = Product::find($id);
    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }
    $request->validate([
        'nama' => 'required',
        'deksripsi' => 'required',
        'harga' => 'required|numeric'
    ]);
    $product->update($request->all());
    return response()->json([
        'status' => true,
        'message' => 'Product berhasil diupdate',
        'data' => $product
    ]);
}
```

Gambar 07: Fungsi Update pada ProductsController

```php
// ◆ DELETE /products/{id}
public function destroy($id)
{
    $product = Product::find($id);

    if (!$product) {
        return response()->json([
            'status' => false,
            'message' => 'Product tidak ditemukan'
        ], 404);
    }

    $product->delete();

    return response()->json([
        'status' => true,
        'message' => 'Product berhasil dihapus'
    ]);
}
```

Gambar 08: Fungsi Destroy pada ProductsController

E. API Documentation

1. Install Scramble dengan menggunakan composer **composer require dedoc/scramble**

2. Masukan ke vendor laravel project anda **php artisan vendor:publish --provider="Dedoc\Scramble\ScrambleServiceProvider"**

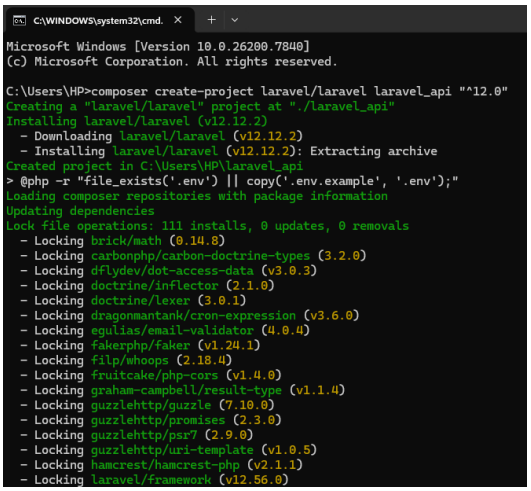
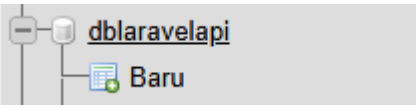
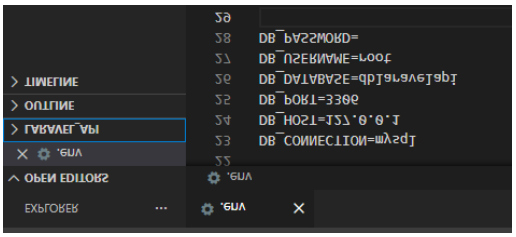
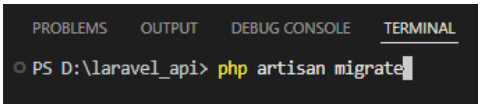
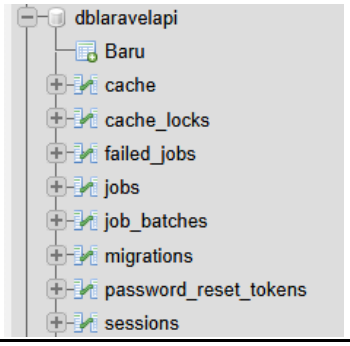
3. Jalankan server (artisan serve)
4. Akses API documentation pada browser ke http://localhost:8000/docs/api
5. Untuk mendambahkan deksripsi setiap API Endpoints bisa menggunakan DocBlock diatas fungsi dalam controller
6. Menambahkan DocBlock (Title dan Description)

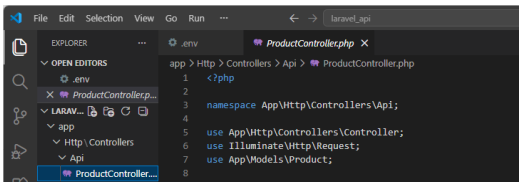
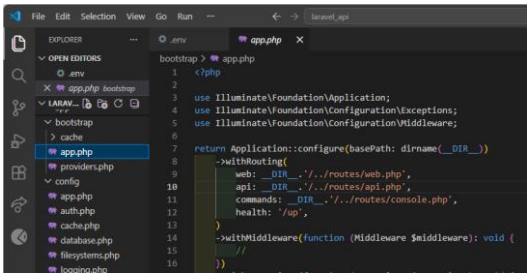
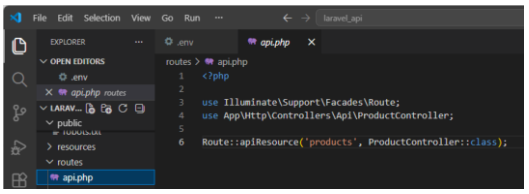
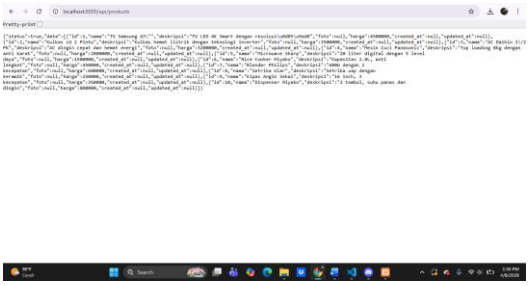
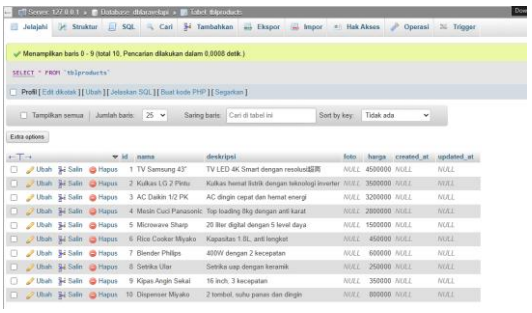
```php
class ProductController extends Controlle
{

/**
 * Ambil semua produk.
 *
 * Endpoint ini mengembalikan seluruh data produk yang tersedia.
 */

    // ◆ GET /products
    public function index()
    {
        $products = Product::all();

        return response()->json([
            'status' => true,
            'data' => $products
        ]);
    }
```

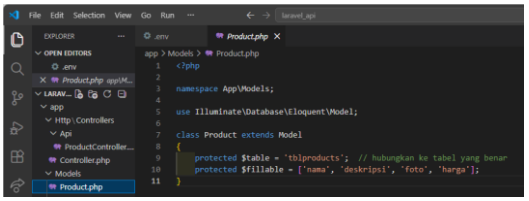
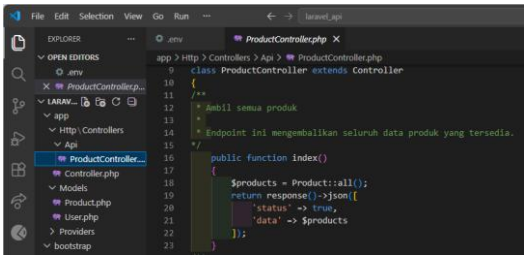
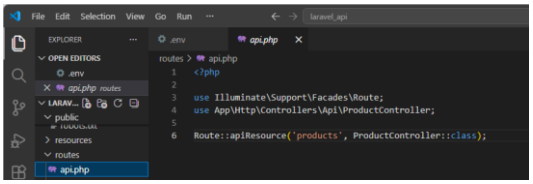
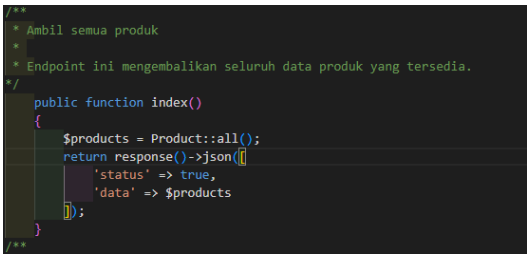
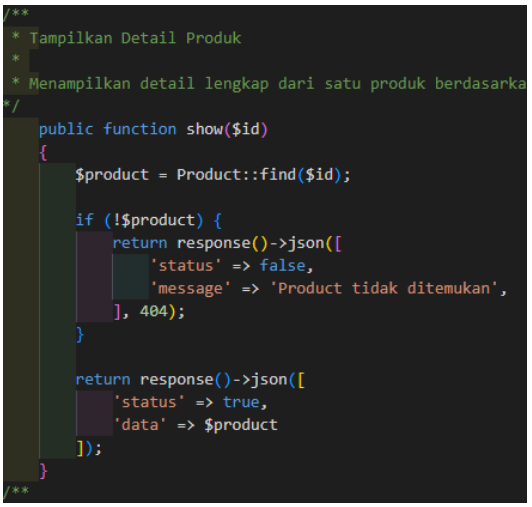
Gambar 08: DocBlock untuk title node index dan deksripsi

7. Refresh browser ke http://localhost:8000/docs/api
8. Tambahkan Title dan Deskripsi pada semua Endpoints

Hasil Pengerjaan

No.	Instruksi	Screenshot	Kendala/Saran
A.	Installasi dan Konfigurasi		
1.	A. Setup Laravel dan Mysql 1.Buat project laravel baru dengan composer dengan nama laravel_api (Gunakan versi 12.x atau yang lebih baru) 2.Mempersiapkan database dan buat database baru bernama dblaravelapi 3.Update .env dan hubungkan dengan nama database dblaravelapi 4.Lakukan migrasi dengan menggunakan php artisan migrate 5.Pastikan laravel sudah terhubung dengan tabel dengan mengecek tabel yang dibuat laravel dalam mysql	     	Tidak ada

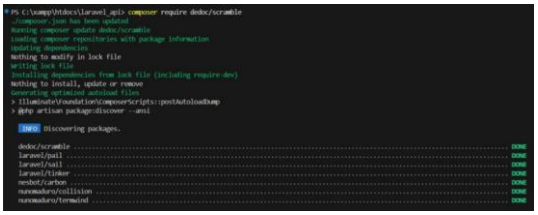
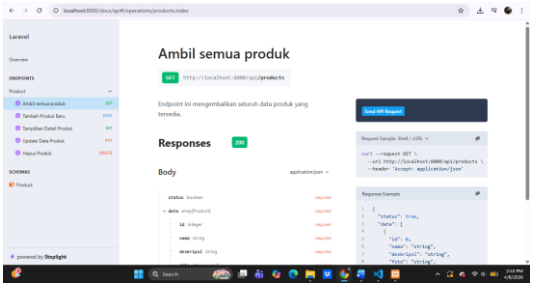
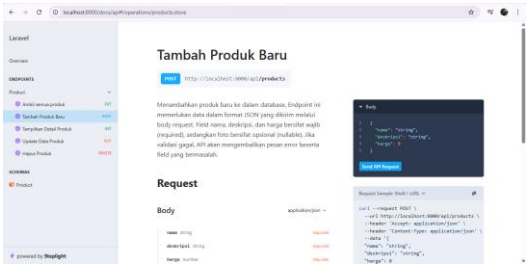
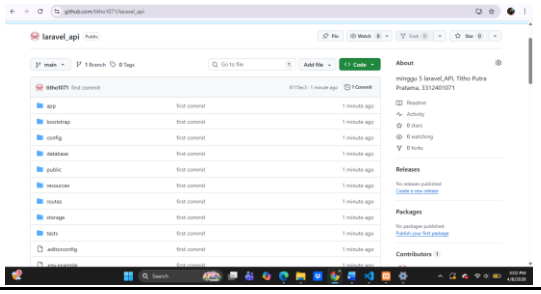
2.	B. Membuat Controller 1.Buatlah sebuah controller dengan nama Api/ProductController php artisan make:controller Api/ProductController (Pastikan ada di dalam folder Api) Pada bootstrap/app.php tambahkan setelah code web: __DIR api: __DIR__.'/../routes/api.php', 2.Tambahkan Routes baru di router/api.php Route::get('/products', [ProductController::class, 'index']); (Apabila file api.php belum ada, clone web.php dan hapus routes “/”) 3. Test API dengan mengunjungi localhost:8000/api/products	   	Tidak ada
3.	C. Gunakan database dan Model 1.Pastikan database sudah terkoneksi dengan laravel. 2.Buatkan sebuah tabel dengan nama tblproducts dengan field (id, nama, deskripsi, foto, dan harga) untuk tipe data menyesuaikan. 3.Tambahkan minimal 10 data untuk produk dengan nama2 product elektronik seperti tv, kulkas, ac, dan lain sebagainya.		Tidak ada

	4. Buat model bernama Product php artisan make:model Product 5.Hubungkan model Product dengan tblproduct dengan mengisi pada class Product 6.Pada Product controller ubah dalam fungsi index, dapatkan data dari model Product Dengan menambahkan $products = Product:all(); Dan lakukan return data => $products	 	
4.	D. Membuat CRUD standar API Resource 1.Ubah api.php dengan api Resource yang sesuai standar laravel 2. Dengan apiResource kamu tidak perlu menambahkan routes satu persatu untuk ProductController seperti create, update, delete, search dan view. Namun ada syarat penamaan fungsi agar routes ini bekerja yaitu index, show, store, update, dan destroy. 3.Ubah ProductController untuk fungsi-fungsi berikut index, show, store, update, dan destroy.	  	Tidak ada

```php
/**
 * Tambah Produk Baru
 *
 * Menambahkan produk baru ke dalam database. Endpoint in
*/
    public function store(Request $request)
    {
        $request->validate([
            'nama' => 'required',
            'deskripsi' => 'required',
            'harga' => 'required|numeric',
        ]);

        $product = Product::create($request->all());

        return response()->json([
            'status' => true,
            'message' => 'Produk berhasil ditambahkan',
            'data' => $product
        ]);
    }
/**
```

```php
 * Update Data Produk
 *
 * Memperbarui data produk yang sudah ada di database. P
*/
    public function update(Request $request, $id)
    {
        $product = Product::find($id);

        if (!$product) {
            return response()->json([
                'status' => false,
                'message' => 'Product tidak ditemukan'
            ], 404);
        }

        $request->validate([
            'nama' => 'required',
            'deskripsi' => 'required',
            'harga' => 'required|numeric',
        ]);

        $product->update($request->all());

        return response()->json([
            'status' => true,
            'message' => 'Produk berhasil diupdate',
            'data' => $product
        ]);
    }
/**
```

```php
 * Hapus Produk
 *
 * Menghapus produk dari database berdasarkan ID yang di
*/
    public function destroy($id)
    {
        $product = Product::find($id);

        if (!$product) {
            return response()->json([
                'status' => false,
                'message' => 'Product tidak ditemukan'
            ], 404);
        }

        $product->delete();

        return response()->json([
            'status' => true,
            'message' => 'Produk berhasil dihapus'
        ]);
    }
}
```

5.	E. API Documentation 1.Install Scramble dengan menggunakan composer composer require dedoc/scramble 2.Masukan ke vendor laravel project anda php artisan vendor:publish --provider="Dedoc\Scramble\ScrambleServiceProvider" 3.Jalankan server (artisan serve) 4.Akses API documentation pada browser ke http://localhost:8000/docs/api 5.Untuk mendambahkan deksripsi setiap API Endpoints bisa menggunakan DocBlock diatas fungsi dalam controller 6.Menambahkan DocBlock (Title dan Description) 7.Refresh browser ke http://localhost:8000/docs/api 8.Tambahkan Title dan Deskripsi pada semua Endpoints	  	Tidak ada
B.	Github dan Viscode		
1.			
2.		https://github.com/titho1071/laravel_api.git	
…			